



Get Your Hands Dirty: Create Business Goals for This System

What are the business goals for this proposed system? Create a few point of view mad libs for this scenario.

Bouncing Bean Grocery is a regional grocery store chain. A few months ago an *Organic Plus!* store opened and *Bouncing Bean* has seen a decline in sales. Hoping to entice customers to their stores, *Bouncing Bean* has hired your team to develop a mobile application in which potential shoppers can create shopping lists, search recipes, and clip e-coupons. *Bouncing Bean* hopes the app will attract customers and provide customer data to drive targeted advertising.

Here are some things to think about.

- Who are the stakeholders? What do they hope to gain?
- Who are the users? What are they trying to accomplish? (Hint: It has nothing to do with software.)
- What's the worst that can happen? Sometimes thinking about failure can help uncover a business goal. People usually want to avoid failures.

Project Lionheart: The Story So Far...

Mayor van Damme gave our product manager a good starting point with his key strategic directives. The first thing she did was verify the mayor's business goals with other stakeholders. After looking at our stakeholder map ([described on page 42](#)), our product manager scheduled meetings with the head of the Office of Management and Budget and two members of city council.

Our product manager led the stakeholder interviews. You observed the interviews to develop a bit more empathy for our stakeholders' needs. Our product manager summarized the business goals using the standard goals template [shown on page 45](#). The whole team reviewed the business goals with Mayor van Damme and the Office of Management and Budget to verify that we understood the business goals correctly. Our product manager added our business goals to the project wiki so that everyone could read them.

While talking to stakeholders about business goals, you heard many requests for features and a few pain points around quality attributes. Since we have a good handle on the business goals, it will be easier for us to focus our design efforts as we dig for additional architecturally significant requirements.

Next Up

Empathy is the engine that drives design. When we know who our stakeholders are and how they hope the software will help them, we will make better design decisions on their behalf. Business goals are a straightforward way to help the team internalize stakeholders' hopes and dreams for the software.

Knowing who the stakeholders are and understanding the business goals is important but doesn't tell us what the software should do or how it is expected to behave. We capture this information as requirements. Architects need different information than what traditional requirements specifications typically offer. In the next chapter, you'll learn how to look at requirements from the perspective of software architecture.

Dig for Architecturally Significant Requirements

Every design discussion starts with *who*, *what*, and *why*. In [Chapter 4, Empathize with Stakeholders, on page 39](#) you learned how to identify *who* is affected by the software system and *why* they care. In this chapter, you'll learn how to define the *what*, the requirements, from the perspective of software architecture.

An *architecturally significant requirement*, or ASR, is any requirement that strongly influences our choice of structures for the architecture. It is the software architect's responsibility to identify requirements with architectural significance. You'll do this by thinking about four categories of requirements:

Constraints Unchangeable design decisions, usually given, sometimes chosen.

Quality Attributes Externally visible properties that characterize how the system operates in a specific context.

Influential Functional Requirements Features and functions that require special attention in the architecture.

Other Influencers Time, knowledge, experience, skills, office politics, your own geeky biases, and all the other stuff that sways your decision making.

Let's take a closer look at these categories of ASRs and learn how to work with stakeholders to define them.

Limit Design Options with Constraints

A *constraint* is an unchangeable design decision you are given or choose to give yourself. Most software systems have only a handful of constraints. All

constraints limit choice, but well-chosen constraints simplify the problem and can make it easier to design a satisficing architecture. Sometimes constraints create a living hell for architects by limiting options so severely we are unable to satisfy other requirements.

Constraints can influence technical or business concerns. Business constraints limit decisions about people, process, costs, and schedule. Technical constraints limit decisions about the technology we may use in the software system. Here are some examples of each:

Technical Constraints	Business Constraints
Programming Language Choice	Team Composition and Makeup
<i>Anything that runs on the JVM.</i>	<i>Team X will build the XYZ component.</i>
Operating System or Platform	Schedule or Budget
<i>It must run on Windows, Linux, and BeOS.</i>	<i>It must be ready in time for the Big Trade Show and cost less than \$800,000.</i>
Use of Components or Technology	Legal Restrictions
<i>We own DB2 so that's your database.</i>	<i>There is a 5 GB daily limit in our license.</i>

Capture Constraints as Simple Statements

To capture a constraint, describe the decision and its origin in a brief statement. There are some constraints for the Project Lionheart system in the [table on page 51, introduced on page 14.](#)

Constraints, once decided, are 100 percent non-negotiable. Be conservative in accepting constraints. There is a huge difference between *this must be done or you will fail* and *this should be done unless you have a good reason not to do it*.

As the software system emerges, design decisions can become constraint-like. Distinguishing between the constraints we created and the ones we were given becomes more difficult as the system grows. Like barnacles on a ship, software slowly gains cruft and becomes less nimble, less clean, less malleable. Eventually, it may become so difficult to amend the architecture that those early design choices become constraints for future designers.

As constraints emerge, be careful to distinguish the constraints chosen for you from the constraints you give yourself. Though it may be difficult, you always have the option of changing a constraining design decision.

Constraint	Origin	Type	Context
Must be developed as open source software.	Mayor van Damme	Business	The City has an Open Data policy and citizens must have access to source code.
Must build a browser-based web application.	Mayor van Damme	Technical	Decreases concerns about software delivery and maintenance.
Must ship by the end of Q3.	Mayor van Damme	Business	Avoids end of fiscal year budget issues.
Must support latest Firefox web browser.	City IT	Technical	Officially supported browser.
Must be served from a Linux server.	City IT	Technical	City uses Linux and open source where possible.

Define the Quality Attributes

Quality attributes describe externally visible properties of a software system and the expectations for that system's operation. Quality attributes define how well a system should perform some action. These *-ilities* of the system are sometimes called *quality requirements*. Here is a list of some common quality attributes from [Software Architecture in Practice \[BCK12\]](#).

Design Time Properties	Runtime Properties	Conceptual Properties
Modifiability	Availability	Manageability
Maintainability	Reliability	Supportability
Reusability	Performance	Simplicity
Testability	Scalability	Teachability
Buildability or Time-to-Market	Security	

Every architecture decision promotes or inhibits at least one quality attribute. Many design decisions promote one set of quality attributes while inhibiting others that are also important! When this happens, we'll trade one quality attribute for another by choosing a structure for the architecture that favors one quality attribute but harms others.

When digging for ASRs, we'll spend most of our time working with quality attributes. Quality attributes are used throughout the design process to guide technology selection, choose structures, pick patterns, and evaluate the fitness of our design decisions.



Joe asks:

Are Quality Attributes Non-functional Requirements?

Traditional software engineering textbooks usually discuss two classes of requirements. *Functional requirements* describe the behavior of the software system. *Non-functional requirements* describe all system requirements that aren't functional requirements, including what we're calling quality attributes and constraints.

When you are designing a software architecture, it's useful to distinguish between functionality, constraints, and quality attributes because each type of requirement implies a different set of forces are influencing the design. For example, constraints are non-negotiable whereas quality attributes can be nuanced and involve significant trade-offs.

Yes, quality attributes are non-functional requirements, but it is strange to use this term to describe them since quality attribute scenarios (sometimes called quality requirements) have a functional piece to them. Quality attributes make sense only in the context of system operation. In a quality attribute scenario, an artifact's response is the direct result of some function.

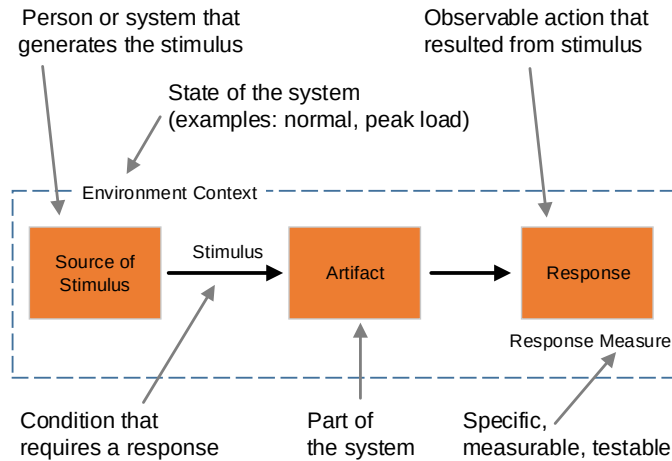
Capture Quality Attributes as Scenarios

A quality attribute is just a word. Scalability, availability, and performance are meaningless by themselves. We need to give these words meaning so we understand what to design. We use a *quality attribute scenario* to provide an unambiguous description of a quality attribute.

Quality attribute scenarios describe how the software system is expected to operate within a certain environmental context. There is a functional component to each scenario—stimulus and response—just like any feature. Quality attributes scenarios differ from functional requirements since they qualify the response using a response measure. It is not enough just to correctly respond. How the system responds is also important. The [diagram on page 53](#) visually depicts the six parts of a quality attribute scenario.

Stimulus The stimulus is an event that requires the system to respond in some way. The stimulus kicks off the scenario and will vary depending on the type quality attribute. For example, the stimulus for an availability might be a node becoming unreachable whereas the stimulus for a modifiability scenario might be a request for a change.

Source The source is the person or system that initiations the stimulus. Examples include users, system components, and external systems.



Artifact The artifact is the part of the system whose behavior is characterized in the scenario. The artifact can be the whole system or a specific component.

Response The response is an externally visible action that takes place in the artifact as a result of the stimulus. Stimulus leads to response.

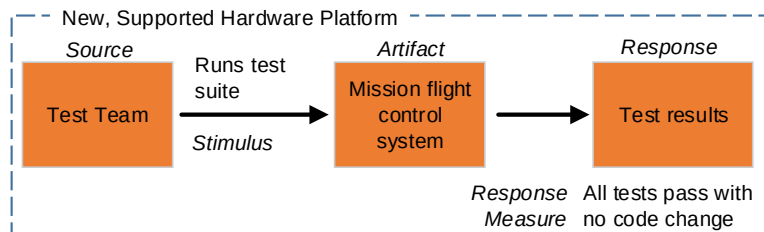
Response Measure The response measure defines the success criteria for the scenario by defining what a successful response looks like. Response measures should be specific and measurable.

Environment Context The environment context describes the operational circumstances surrounding the system during the scenario. The environment context should always be defined even if the context is *normal*. Abnormal contexts, such as peak load or a specific failure condition, are also interesting to consider.

Here is an example portability scenario for an interplanetary robotic explorer based on examples from [the NASA Jet Propulsion Laboratory \[WFD16\]](#).

Portability Scenario for a Mars Rover (via NASA JPL)

Raw scenario: Processors and platforms are typical variation points project to project. Enabling projects to select processors and platforms with minimal effects to applications allows for system optimization.



Notice that the *raw scenario* in our example doesn't mention specific response measures. Raw scenarios are simple descriptions that form the basis for more precise quality attribute scenarios. We call them raw because they need further cooking to become a good scenario. Think of a raw scenario as the start of a conversation.

Specifying all six parts of a formal quality attribute scenario is not always necessary. You can often get by with a simple statement that includes the stimulus, source, response, and response measure. Add the environment whenever the scenario does not describe a *normal* environmental context.

Here are some quality attribute scenarios for the Project Lionheart case study:

Quality Attribute	Scenario	Priority
Availability	When the RFP database does not respond, Lionheart should log the fault and respond with stale data within 3 seconds.	High
Availability	A user's searches for open RFPs and receives a list of RFPs 99% of the time on average over the course of the year.	High
Scalability	New servers can be added during a planned maintenance window (less than 7 hours).	Low
Performance	A user sees search results within 5 seconds when the system is at an average load of 2 searches per second.	High
Reliability	Updates to RFPs should be reflected in the application within 24 hours of the change.	Low
Availability	A user-initiated update (for example, starring an RFP) is reflected in the system within 5 seconds.	Low
Availability	The system can handle a peak load of 100 searches per second with no more than a 10% dip in average response times.	Low
Scalability	Data growth is expected to expand at a rate of 5% annually. The system should be able to grow to handle this with <i>minimal</i> effort.	Low

A good-quality attribute scenario communicates the intent of the requirement so anyone can understand it. Great scenarios are precise and measurable. Two people who read the same quality attribute scenario should come away with the same understanding of the system's *scalability* or *performance* or *maintainability*.

Strive for Specific and Measurable Response Measures

To create a response measure, start by estimating potential values based on your own experience. Use a straw man to kick off a conversation with stakeholders (see [Activity 9, Response Measure Straw Man, on page 219](#)). What if it took nine months to migrate the system to a new microcontroller platform, would that work? How about six months? Eventually, you'll find a response measure that resonates with stakeholders.

Good response measures are testable. Early in the system's life, the architecture might exist only on paper, but it's just a matter of time before you have a running system. If you can't write a test using your scenario, then the scenario does not have a specific, measurable response measure.

Choose Appropriate Response Measures

I was once on a team responsible for building a simulation testbed for a military combat system. The purpose of the testbed was to connect a dozen military bases across the world so we could play simulated war games. To run a test, we would play a scenario that generated fake aircraft. The hardware and software at each site would detect the simulated aircraft, and the combat systems would process the sensor data as if it were from the real world.

The simulation testbed had extremely aggressive latency requirements. If all sites did not receive the same simulation data within a narrow window of time, it would seem as if aircraft were appearing and disappearing from the sky. Even worse, aircraft might be visible only to some sites in the network. Too much latency would invalidate the system tests.

After crunching some numbers, we determined our testbed would need to transfer data faster than the speed of light for everything to work. The performance and availability response measures were nowhere near reality. Once quantum entangled networks become viable, it will be interesting to revisit this problem.



Get Your Hands Dirty: Refine These Notes into Quality

Attribute Scenarios

During a meeting, Project Lionheart stakeholders shared the following statements. For each statement, identify the quality attribute and create a formal, six-part quality attribute scenario.

- There's a small number of users, but when a user submits a question or problem we need to be able to respond quickly, within a business day.
- Releases happen at least once a month. Ideally, we'll ship code as it is ready.

- We need to verify that the RFP index is built correctly. The verification should be automated.
- We need a new, permanent dev team to come up to speed quickly after the current team of contractors we've hired leaves.

Here are some things to think about:

- What quality attribute is suggested by each statement? It's OK to make up an *ility* if it helps describe the concern effectively.
- Are there implied responses or response measures?
- What missing information can you fill in based on your own experiences?

Look for Classes of Functional Requirements

Functional requirements, often captured as use cases or user stories, define the software system's behavior but are only sometimes interesting when designing the architecture. All functional requirements are essential to the success of the software system, but not all system features have architectural significance. When a functional requirement drives architectural decision making, we call it an *influential functional requirement*.

Influential functional requirements can be referred to as *architecture killers*. If your architecture doesn't allow you to implement one of these high-value, high-priority features, you'll be forced to raze your architecture and start over.

Identifying influential functional requirements is equal parts art and science. It becomes easier with experience. Here's how I do it:

1. Start with a notional architecture sketch that summarizes your current thinking about the architecture.
2. Identify general classes of requirements that represent the same type of architectural problem.
3. For each problem class identified, walk through the notional architecture and show how to achieve each requirement group. If it is not immediately obvious how you would implement the feature based on the known coarse-grained requirements, it might have architectural significance.

The goal of step two is to reduce a giant list of functional requirements down to a small number of representative categories. Here are a few strategies:

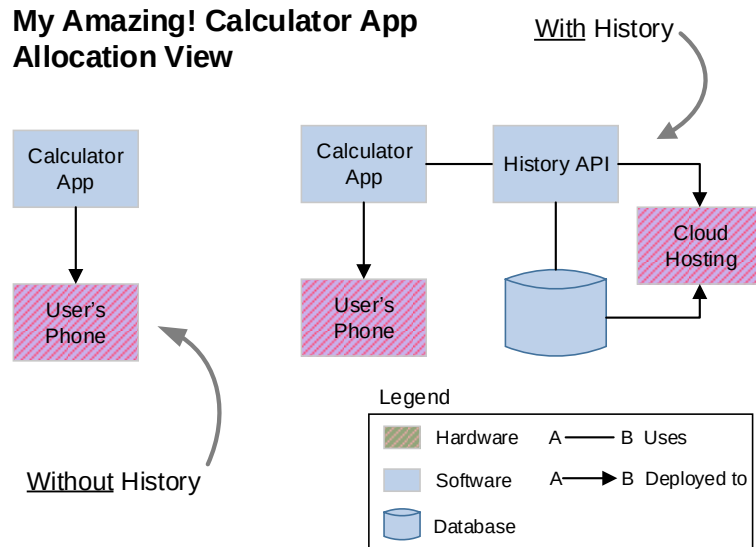
- Look for functional requirements that might be implemented within the same architectural elements. For example, features that require

persistence go in one group whereas features that require user interaction go in another.

- Look for functional requirements that seem difficult to implement. These could be significant to the architecture.
- Look for high-value, high-priority functional requirements.

Here's an example. Recall the [simple calculator example on page 10](#). Adding two numbers together is an important functional requirement but has little influence on the architecture, so let's spice it up a bit. Here's a new feature: *as an Adder User I can review my addition history even if I've lost my phone*. "People love looking at past stuff they've done," the marketing team assures us. "It's going to be A-Mazing!"

Historical information? OK, no problem. We can save the user's actions to a local database. Wait... even if they've lost their phone?!?! Now we need a remote database server, which opens up a ton of new questions. What happens when the user's phone is offline? What about availability? Scalability? Hosting costs? Syncing the app when the schema changes? The list goes on.

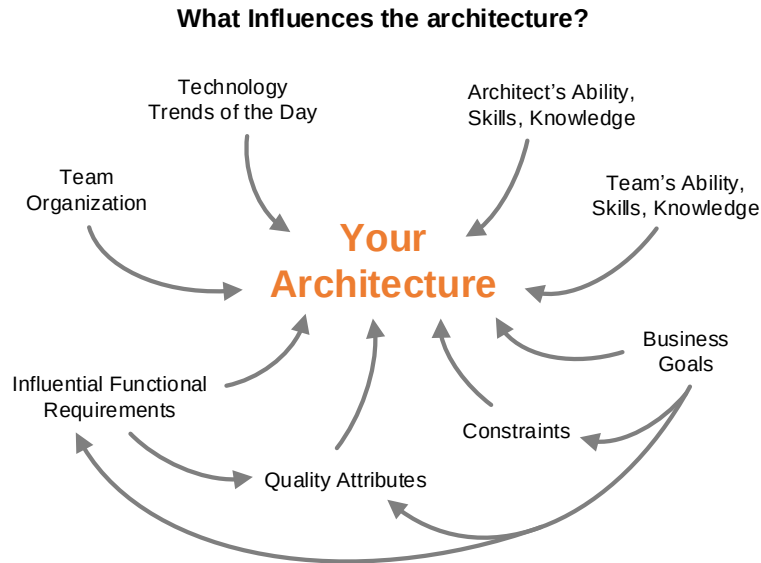


That one seemingly innocent feature request introduces a swirl of complexity. In our simple calculator example, we can reduce any mathematical operation to the same general problem. To solve the newly requested history feature, we need remote storage. This one feature takes the architecture in a new direction that other functional requirements did not.

Reference any influential functional requirements in your architecture documentation but avoid duplicating the requirements engineering effort. Our goal is to call attention to critical features that influence our decision making.

Find Out What Else Influences the Architecture

In addition to ASRs, there is a slew of other factors that will affect the architecture both directly and indirectly. Here is a list of some of the factors that might influence the architecture:



Your skills and experience as an architect determine how you approach design and the architecture options available to you. Your knowledge and your team's knowledge of technology defines your design vocabulary. If all you know is Ruby on Rails, then the chances are good you'll find some way to wedge it into the architecture. When all you have is a hammer, you will find plenty of nails to hit.

Architecture always seems to follow hot technology trends. As new hardware, software, and design paradigms emerge, some will permanently alter the software engineering landscape. Others might just be marketing veneer on old ideas. There's a good chance your architecture is already proudly sporting the design equivalent of a mullet hairstyle.

Learn to Live with Conway's Law

How your team is organized and prefers to collaborate influences the architecture design. Conway's Law, coined by Melvin Conway in 1967 and popularized by

Fred Brooks in the [Mythical Man Month \[Bro95\]](#), describes the relationship between team organization and architecture.

...organizations which design systems...are constrained to produce designs which are copies of the communication structures of these organizations.

If you have three teams, you'll end up with three components. Communication boundaries among people manifest as element boundaries in the architecture. Conway's Law works both ways. Communication paths designed into the architecture will also influence how you organize your teams. If you want to design the best software possible, then you must be prepared to reorganize your team.

Other influencers are usually only recorded as part of the rationale for design decisions. So many things can influence the architecture that it is practically impossible to document all the potential influencers prior to making design decisions.

Dig for the Information You Need

Architecturally significant requirements are hidden all around us. You'll find ASRs in user stories, implied by a manager's request, and hinted by stakeholders who know what they want but don't quite know how to explain it.

The product backlog contains a treasure trove of ASRs. Quality attributes are implied or assumed in nearly every functional requirement. Sometimes a user story will plainly describe response times, scalability needs, or how to handle failures. Highlight these details as quality attribute scenarios lest they get lost in the feature backlog.

Talk to stakeholders. Find out what worries them. Ask stakeholders what excites them. Share the risks and open questions you see. Here are some additional methods you can use to dig out interesting ASRs:

- Use [Activity 3, Goal-Question-Metric \(GQM\) Workshop, on page 199](#) to connect business goals and quality attribute response measures with concrete data requirements.
- Use [Activity 4, Interview Stakeholders, on page 202](#) to uncover quality attribute scenarios and constraints. Interviews work especially well with technical stakeholders.
- Use [Activity 5, List Assumptions, on page 205](#) to flush hidden requirements into the open.
- Use [Activity 7, Mini-Quality Attribute Workshop, on page 210](#) to quickly and effectively define high-priority quality attribute scenarios. This workshop

works for nearly any kind of project and with stakeholders of different skills and backgrounds.

- Use [Activity 24, Inception Deck, on page 269](#) as a checklist for kicking off a new project. Architecture is the main topic for several slides in the inception deck.

Build an ASR Workbook

Once you’ve identified requirements with architectural significance, record them in an *ASR Workbook*. At the beginning of a new software system, the ASR Workbook is a living document and changes rapidly. As the architecture coalesces, you’ll edit the workbook less frequently but reference it more often. Executable tests and source could eventually supplant portions of the ASR Workbook as a source of truth, though the document will remain an important historical record.

The ASR Workbook provides context and information for programmers, testers, and of course, architects. The more people who understand the ASRs, the less architectural oversight will be required.

Here is a sample ASR Workbook outline. Use the outline as a checklist for planning requirements elicitation.

Sample ASR Workbook Outline
Purpose and Scope
Intended Audience
Business Context
Stakeholders
Business Goals
Architecturally Significant Requirements
Technical Constraints
Business Constraints
Quality Attribute Requirements
Top Scenarios
Influential Functional Requirements
Top Users or User Personas
Use Cases or User Stories
Appendix A: Glossary
Appendix B: Quality Attributes Taxonomy



Thijmen says:

Learn to Be an Active Listener

by Thijmen de Gooijer, IT Architect

Understanding your stakeholders and their goals is the first step to successfully delivering value with software. Taking someone else's perspective and showing empathy will help you understand their expectations of your software. Technical training and experience then turn requirements into implementable ideas. However, you will need excellent communication skills to get developers and management to share your vision and turn ideas into code.

One of the most useful communication skills I had to learn is active listening. Hearing what someone says is only the first step. You also have to understand it. Here is a surprisingly challenging exercise that I learned during a course on communication. You can try this exercise with a partner.

Person A tells a story to person B, for example about an achievement or describing a problem they solved. The trick is that person B is not allowed to say a word until person A indicates they are finished. Only then is person B allowed to ask questions to increase understanding of what person A said. The questions cannot be covert feedback or critique. Person B's goal is to reach an understanding. Now reverse roles to experience the other side of the relationship.

Imagine how hard this exercise can be! You probably know a colleague who talks a little bit too much or a shy and quiet intern. How would you ensure that you understand their requirements?

Writing down directly what a stakeholder tells you probably won't lead to an implementable requirement. Human language is messy, complicated, and full of culturally loaded messages—nothing like COBOL, Java, PHP, or Python. Culture does not only differ between countries or religions. Cities, companies, schools, and sports clubs have cultures, which influence how people communicate.

As an active listener, you need empathy to put words into their cultural context and understand them. Remain quiet, don't judge, and ask questions to help you understand.

Communication skills are hard to learn from books, yet I recommend two to assist you on your way to becoming an amazing software architect. [How to Win Friends and Influence People \[Car09\]](#) by Dale Carnegie is a classic book that gives actionable guidance on how to build better relationships with people. In [Culture Clash 2: Managing the Global High Performance Team \[Zwe13\]](#) Thomas D. Zweifel provides an easy-to-understand framework for identifying and overcoming cultural differences.

Use the ASR Workbook to introduce architectural concepts to your team and stakeholders. Briefly teach readers what business goals, constraints, quality attributes, and influential functional requirements are, and they'll have a finer appreciation for the information in the document.

Project Lionheart: The Story So Far...

During a user experience workshop facilitated by our product manager, you discover dozens of new features. You add these features to the product backlog and make a note in the few functional requirements that seem to have architectural significance. You also jot down several potential constraints to verify with stakeholders.

A few days after the requirements workshop, you facilitate a mini-quality attribute workshop with several stakeholders. During the workshop, you elicit and prioritize nearly two dozen quality attribute scenarios. You don't formally record all the concerns raised during the workshop, but you collaborate with participants to refine the top seven highest-priority scenarios.

Up to this point, our primary focus was to understand the problem. We made several artifacts so we could share what we know about the problem with our stakeholders. You uncover a lot in a few short days on site. Looking at the team's list of open questions, you think we have enough information to embrace the explore mindset and start choosing structures for the architecture.

Next Up

Many different architectures could implement the same set of features. Features alone are not enough information for us to design a software system. It's the architecturally significant requirements, especially quality attributes, that drive architectural decision making.

Solutions flow from our understanding of the problem. We do not need to wait until we understand everything about the problem before thinking about potential solutions. We'd never build anything if we waited to define the whole problem! As you explore solutions, you'll uncover new insights about the problem. Discovering there is more to the problem than you knew is natural and expected. In the next chapter, you'll learn how to use what we currently know about the problem to explore design options and make decisions.

Understand	Explore
Evaluate	Make

CHAPTER 6

Choose an Architecture (Before It Chooses You)

Every software system has an architecture. That doesn't mean you'll end up with one you want. When you leave design decisions to fate, there is no telling what fate will deliver. Actively making decisions about how to organize the software system significantly increases our chances of getting an architecture that meets our needs.

Designing software architecture is all about making decisions under uncertainty. Design decisions are loaded with *trade-offs*, decisions that force us to compromise—give up something good to avoid something bad, or accept something bad to get something better. If we make acceptable trade-offs, then we'll achieve our architecturally significant requirements and help our stakeholders reach their business goals. Yay!

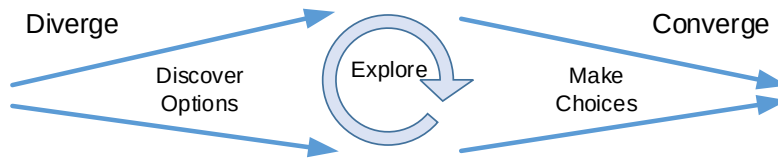
In this chapter, you'll learn how to choose structures for the architecture by using architecturally significant requirements to drive your decision making.

Diverge to See Options, Converge to Decide

Making a decision implies we've seen multiple options from which to choose. If there is only one option, then we didn't decide anything; the decision was made for us. To ensure we see many options, we need to explore the design space.

Design exploration is an iterative journey of divergence and convergence. Once we've identified a problem, we *diverge* our thinking and generate design alternatives that can solve that problem. Once we have a few options on the

table, we'll *converge* our thinking by building consensus and eliminating options that are a poor fit for the current problem.



Human brains crave options. Our confidence in a decision increases after seeing multiple alternatives. Unfortunately, there isn't time to explore every possible option and all aspects of a software system's design. Architects need to focus on and champion quality attributes, structural organization, and design decisions that will influence these things.

Explore the Architecturally Significant Things

Grady Booch has said, "All architecture is design, but not all design is architecture."¹ As you learned in [What Is Software Architecture?, on page 7](#), a system's software architecture is the set of significant design decisions about how the software is organized to promote desired quality attributes and other properties. Architects must explore these significant design decisions and actively choose how to organize the software to achieve desired quality attributes.

Here are areas of a software system's design architects will typically explore:

Explore elements and their responsibilities to determine the general composition of structures in the architecture. Recall from [Define the Essential Structures, on page 7](#) that structures in the architecture are made up of elements. In a well-designed architecture, every element has clear responsibilities. Any element without a well-defined responsibility should be eliminated. Exploring design options requires that we explore combinations of elements with varying responsibilities.

Explore relations and their interfaces to determine how elements interact with one another. Relations describe how two elements in the architecture work together to accomplish a task. A component's interface is one example of a relation. Both the communication mechanism (for example, HTTP, TCP, or shared memory) and the rules for communication (such as APIs, response objects, or required data) define the interface. The rules

1. Grady Booch. *Abstracting the Unknown*. SATURN 2016. <http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=454315>

governing interfaces and element communication are inherently architectural, at least up to a point. We can defer some details—such as method names and sometimes the fields returned in a response—to downstream designers.

Explore the domain to understand the world the architecture models.

Every problem has its own terminology and concepts, which describe the world in which it exists. The concepts from the domain, be they objects or events, must be accounted for somewhere in the architecture. The better we understand the problem domain, the better we'll partition elements and assign responsibilities to them in the architecture.

Explore technology and frameworks to bootstrap promoting quality attributes.

Modern software development technologies are loaded with architecture assumptions. Frameworks, middlewares, libraries—any off-the-shelf technology—comes with attitude. The technology will tell you how and when to use it. Opinionated technologies force decisions on to the architecture.

When the technology aligns with our needs, then life is rainbows and unicorns. When our needs fall outside the bounds of what the tech thinks we need, then prepare for a battle royal between you and the framework.

Explore construction and deployment methods to ensure the architecture can be shipped.

How we design the architecture influences how the software is constructed and deployed. If we desire continuous delivery, if we want to have multiple developers working in parallel, if we require the use of specific testing strategies, then we must design the architecture to support these requirements.

Explore past designs to gain perspective and guide decision making.

All design is redesign. Most architecture explorations start by looking at what we already know about how to design software. We can codify design knowledge as a rule of thumb or a documented pattern. Knowledge can come from your own experience or as legends passed from architect to architect over the ages.

Since we want to create a clear connection between our design decisions and stakeholders' needs, we'll use the categories of architecturally significant requirements from [Chapter 5, Dig for Architecturally, on page 49](#) to organize our approach to exploration and decision making.



Len says:

Don't Forget Deployment

By Len Bass, independent consultant and co-author of *Software Architecture in Practice* [BCK12], *Documenting Software Architectures: Views and Beyond* [BBCG10], and *DevOps: A Software Architect's Perspective* [BWZ15]

One of the most easily overlooked items when designing a system or service with multiple instances during execution is deployment. There are two basic methods for deploying a new version of a service with multiple instances: red/black or rolling upgrade.

A red/black deployment (some names use different colors like blue/green) allocates sufficient virtual machines for all instances of the new version, deploys the new version into those instances, and then switches to use the new instances. A rolling upgrade will upgrade one instance at a time.

In either case, there are possibilities of inconsistencies. For example, suppose you have a chain of services—Service A depends on Service B, which in turn depends on Service C. Now one of your developers deploys a new version of Service B. This new version may change the syntax or semantics of the interface. What happens when Service A invokes Service B and gets an incorrect error because the semantics of an interface has changed? What happens when the new version of Service B assumes a new version of Service C, and the new version of Service C has yet to be deployed?

If you are deploying new versions using a rolling upgrade strategy, then it is possible that two different versions of Service B with different interfaces will simultaneously be executing.

There are a collection of techniques used to overcome these inconsistencies—enforcing backward compatibility, using feature toggles, gracefully handling unknown responses from a dependent service—but the first step is recognizing that deployment and deployment strategies can cause inconsistencies when multiple instances of a service are being run.

Accept Constraints

In *Limit Design Options with Constraints*, on page 49 you learned that constraints are predetermined design decisions that cannot be changed. Recall there are two types of constraints: technical and business. Technical constraints limit your technical options whereas business constraints focus on people, process, cost, and schedule.

We have no choice but to embrace technical constraints and incorporate them into the architecture. If we agree that *the system must be written in .NET*, then there is no point lamenting over the loss of your favorite Java framework.

Business constraints are a bit more nuanced. You also have to accept these, but their impact on the architecture is not always evident. For example, say you've agreed to a business constraint that the system must be ready in time for a trade show at the end of July. You might consider several architectural decisions to satisfy this constraint:

- Choose a pattern that promotes concurrent development effort.
- Choose a pattern that promotes incremental delivery.
- Choose technologies the team is familiar with to reduce risks.
- Choose technologies that support automation and development speed.
- Choose to skip planning, accept technical debt, and build a ball of mud.
- Choose combinations of all these ideas.

Business constraints can also be satisfied outside the architecture—for example, by emphasizing craftsmanship and early testing, or by using a subcontractor with lower hourly rates.

Remember that early design decisions can become constraining, but they are not constraints. Like the load-bearing walls of a house, these early design decisions hold everything else in place. Moving a load-bearing wall in a house might be costly and challenging, but it is technically possible. Always distinguish between the constraints you are given and the constraints you give yourself.

While constraints strongly influence the architecture, most of our design decisions (and trade-offs) in the architecture will focus on promoting desired quality attributes.

Constraints Can Have Far-Reaching Consequences

Years ago a start-up I worked for was acquired by a much larger and more risk-averse corporation. Shortly after the acquisition, our new corporate legal team informed us about some new policies. One of the big policy changes required that we no longer use open source software released under certain licenses.

This new constraint, handed down by the legal team, created nearly a year of new development work. The lesson: Stakeholders don't always understand how a constraint might impact the architecture. When a constraint causes significant pain, talk with your stakeholders about the ramifications of their requirements. In our case, we couldn't sway the lawyers, but sometimes hard constraints become soft when the impact is understood.

Promote Desired Quality Attributes

Selecting structures for the architecture is like making a smoothie. Smoothies (like software) are tasty but difficult to make well. While many things must go right to create great software, there is only one thing you need to get right for a great smoothie: use the proper blender. OK, there are lots of things that can go wrong with smoothie making too (berry seeds, ugh!), but the blender is a crucial smoothie-making tool.

You'd think picking a blender would be easy. It's not. Do you want one that's easy to clean? Something that stores easily and fits on your countertop? Something quiet? Or powerful? Or portable so you can blend on the beach? We can express these needs as blender quality attributes.

Here are three types of blenders capable of making smoothies. Each blender is designed to promote different quality attributes and, as you can see, no two blenders are the same.

Standard
Blender



Hand
Blender



Chainsaw
Blender



Chainsaw Blender photo credit: Mike Warren

The standard blender is dishwasher safe and has a sturdy base for sitting on a kitchen counter top. But it requires electricity, so you're limited to the kitchen. The battery-powered hand blender is small, portable, and easy to clean, but trades power for portability. Finally, the gas-powered chainsaw blender has the best power and portability.² Too bad the 37cc two-stroke, motorcycle-throttle-controlled chainsaw engine is a tad loud and emits an exhaust unsafe for indoor use.

2. Yes, this is a real, working blender. Instructions for making your own can be found at <http://www.instructables.com/id/Chainsaw-Blender/>.

Here is a summary of how well our top-quality attributes are promoted by each blender:

	Standard Blender	Hand Blender	Chainsaw Blender
Cleanability	Neutral	Positive	Neutral
Counter top-ability	Positive	Negative	Strongly Negative
Quietness	Neutral	Positive	Strongly Negative
Power	Neutral	Negative	Strongly Positive
Portability	Strongly Negative	Positive	Strongly Positive
Safety	Neutral	Neutral	Negative

Each blender performs the same basic function (blending). They have interchangeable parts. For example, the same glass pitcher works with the standard and gas-powered chainsaw blenders. In addition to the blender quality attributes, the designers considered costs, manufacturing techniques, interfaces with external systems (human and machine), and other properties. The structures we see in the final designs were chosen to promote properties the designers highly valued.

Just like the blenders, architects choose structures to promote quality attributes in the software system. The most common way to select structures is by exploring patterns. Remember, all design is redesign! Find patterns that promote desired quality attributes and use those patterns as a starting point for the architecture.

Explore Patterns with Quality Attributes in Mind

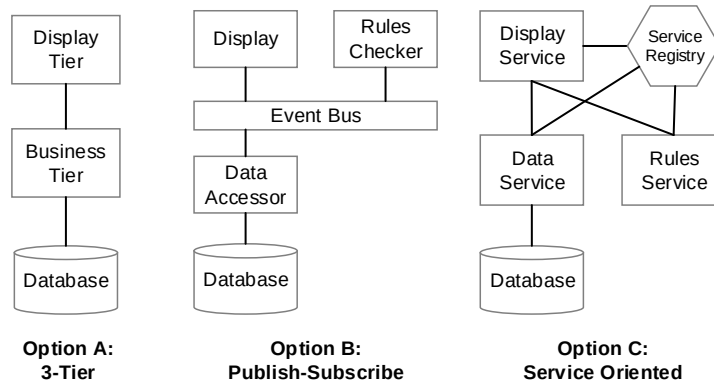
We'll explore architecture patterns in greater detail in [Chapter 7, Create a Foundation with Patterns, on page 79](#). For now, let's see a simple example of how we can use quality attributes to choose an appropriate pattern.

Say we want to build a web-based, data-driven application. What patterns would you choose for this application? There are three decent options: 3-tier, publish-subscribe, and service-oriented as shown in the [figure on page 70](#).

Option A: 3-Tier Pattern. [Introduced on page 92](#), each tier is responsible for different application concerns. For a web application, the display tier renders the UI, the business tier operates server side to verify business rules, and the database stores data.

Option B: Publish-Subscribe Pattern. [Introduced on page 88](#), each element publishes messages to an event bus. Interested components may subscribe

Which architecture pattern would you choose for a data-driven web application?



to message types. Depending on the rules of the message system, events might not be delivered in order and delivery might not be guaranteed.

Option C: Service-Oriented Architecture Pattern. [Introduced on page 86](#), services register with a central registry so that callers can find them. Components look up and call those services directly and the service responds with the requested information—or doesn't if something goes wrong.

Each of these patterns promotes and inhibits different quality attributes. Which one would you choose?

The 3-tier pattern is ideal in many situations. It's easy to test, easy to deploy, and easy to describe. This simplicity comes at a cost. The multi-tier pattern does not promote quality attributes such as scalability and availability. Depending on other quality attribute scenarios, this pattern may not address all our needs without augmenting the architecture with other patterns.

The publish-subscribe pattern is highly modifiable and extremely flexible. This flexibility makes it easy to build loosely coupled systems. While this flexibility and modifiability are attractive, there is a downside. Message order matters to the events in our data-driven application, but the publish-subscribe pattern alone can't guarantee message order. With the right message bus technology, we might make this pattern work, but it feels awkward.

Like the other patterns, service-oriented architecture is modifiable, flexible, and testable. Service-oriented systems are also scalable and promote availability easier than our other options. It's also the most complex of the three patterns under discussion and has the steepest infrastructure curve. Depending on our specific quality attribute scenarios, this pattern could be overkill.

We could successfully implement the functional requirements with any of these patterns. The quality attributes are what really drives our decision making. With quality attributes, there is rarely a single *right* or *wrong* design, only designs that are *better* or *worse* relative to desired system properties. To make a decision we need to do some analysis.

Create a Decision Matrix

The *decision matrix* is a simple tool for summarizing the trade-off analysis among architecture design options. Use it to make decisions about any architectural choice from patterns to functional responsibilities to technology choice. Here is an example:

	Option A	Option B	Option C
Property 1	Inhibits	—	
Property 2	Promotes	+	
Property 3	Neutral	0	

To use a decision matrix, list properties you plan to use for the analysis in the first column of a table. Each row represents a particular property. Each column represents your analysis of a design option.

Summarize the results of your analysis with an easy-to-read notation such as words, arrows, symbols, or colors. The idea is to create a visual representation that shows how each option influences properties you think are valuable. Here is an example:

Strongly Promotes	The design option actively helps you to achieve the system property.
Promotes	The design option allows you to achieve the system property.
Neutral	The design option neither helps nor hurts the system property.
Inhibits	The design option makes achieving the system property slightly more difficult.
Strongly Inhibits	The design option makes it costly or significantly difficult to achieve the system property.

The best decision is often obvious once it's in the matrix. Use a scale like the following to summarize the analysis of each architecture design option. Immediately eliminate any design option that prevents you from achieving a required system property.

We already saw a [decision matrix for smoothie blenders on page 69](#). Let's create another decision matrix, this time for the Project Lionheart patterns [discussed on page 69](#). Here is a decision matrix, which shows some of the analysis for the quality attribute scenarios we defined in [Capture Quality Attributes as Scenarios, on page 52](#):

Project Lionheart Decision Matrix

	3-Tier	Publish - Subscribe	Service Oriented
Availability (Database unavailable)	+	○	+
Availability (Uptime requirements)	○	○	○
Performance (5-second response time)	○	—	+
Security	○	—	○
Scalability (5% annual growth)	○	○	+
Maintainability (Team knowledge)	+	—	○
Buildability (Implementation risks)	++	—	--

Legend

Strongly Promotes	++	Strongly Inhibits	--	Neutral	○
Promotes	+	Inhibits	—		

Looking at the decision matrix, which pattern would you choose? Are there other factors not captured in the matrix that might influence your final decision?

The work that goes into creating the matrix is more important than the matrix itself. The decision matrix is a convenient way to summarize findings and facilitate discussions with stakeholders. We want stakeholders to have a robust discussion about trade-offs among design decisions. Be prepared to explain the scores in the matrix.

Using numbers in the matrix is tempting. Don't. Numbers give a false sense of confidence and precision in the analysis. Eventually, someone will try to adjust scores with weighted stakeholder preferences, sum the columns, consider aggregate averages, or some other dreadful idea. It's just bad news.

See [Activity 32, Decision Matrix, on page 292](#) for further details on this method. An alternative method for helping discuss trade-off priorities is the trade-off sliders activity [described on page 192](#).

Assign Functional Responsibilities to Elements

Every element in the architecture has a job to do. As we choose structures, we'll assign specific functional responsibilities to each element so we can achieve all the essential functional requirements.

Let's look at an example from our case study system, Project Lionheart. Here are some functional requirements gleaned from interviews with people from the Office of Management and Budget.

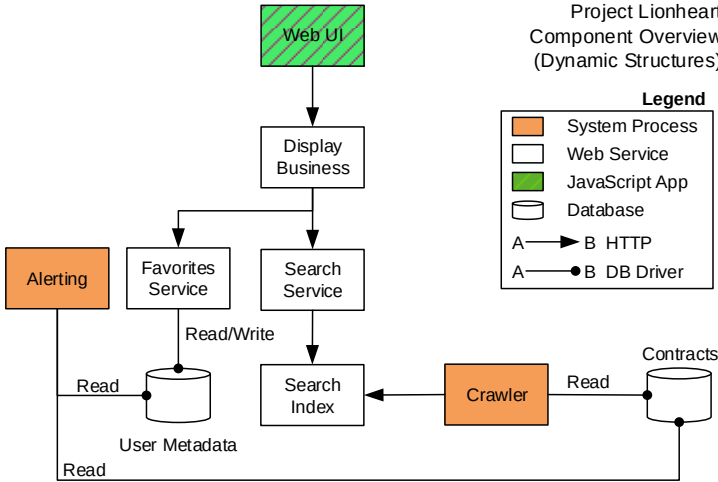
An Office of Management and Budget user can:

- Search existing and past city contracts
- Paginate through all results
- View basic information about a company including the name, phone number, address, and list of past and active contracts
- View basic information about a contract including the type, status, expiration date, PID, bidding companies, and who won the contract
- Subscribe to receive alerts about contract updates

Several responsibilities are implied by these functional requirements, in addition to the things directly mentioned:

- Since users can search, this suggests it must be indexed.
- To show contract and company information it must be stored.
- Subscriptions require that the system stores email addresses.
- To alert users about changes implies something can recognize when a change has happened.

Here is one view of a set of elements that will allow us to achieve these functional requirements shown in the [figure on page 74](#).



Here is an *element responsibility catalog* for this diagram:

Element	Responsibility
Web UI	Renders a user interface for the user in their web browser, handles user interactions.
Display Business	Authentication and authorization, proxy for other backing services, verifies business logic for application use.
Search Service	Core processing for query parsing, search, pagination, filtering.
Favorites Service	Normalizes tags, writes favorites to persistent storage.
Alerting Service	Scheduled to look for recent changes, sends email based on subscriptions stored in user metadata database.
Crawler	Reads data from the Contracts DB, transforms it for search, uploads to index.
User Metadata DB	Persistent storage for user subscriptions and other user-added content.
Search Index	Optimized representation of contract data designed for search. All contract data to be displayed in the UI is searchable, sortable, and stored.
Contracts DB	Persistent storage. System of record for city RFP data.
HTTP Relation	Communication among services over standard HTTP protocols. APIs are assumed to be RESTful.
DB Driver Relation	Native driver/client for the to-be-selected database.

The element responsibility catalog describes the essential duties each element in the architecture has the authority to perform. We created this element responsibility catalog by running down the list of the known influential functional requirements and ensuring each function was owned by one and only one element. Also, each element in the architecture should have at least one function for which it is responsible; otherwise, that element is without purpose.

Influential functional requirements make for a great checklist when assigning responsibilities to elements. One approach for identifying responsibilities is to model the system with component responsibility collaborator cards as [described on page 232](#).

Design for Change

So far in this chapter, you've learned how to explore options and make decisions using your understanding of the ASRs. Making significant design decisions is supremely important for having a robust architecture, but if there is one constant in software, it's change.

All great architectures account for the inevitability of change. We design for change by choosing when to make a decision and by moving design decisions out of the architecture.

Defer Binding Decisions until the Most Responsible Moment

Making a decision that cannot be easily reversed—an architectural decision—is a big deal. One strategy for avoiding dead ends and wrong turns is to defer making binding decisions for as long as responsible. Delaying design decisions until they must be decided creates time for research and exploration.

In *Lean Software Development: An Agile Toolkit for Software Development Managers* [PP03], Mary and Tom Poppendieck introduced the idea of the *last responsible moment*, the time when a decision must be made to avoid losing important design alternatives. Instead of thinking about the last responsible moment, we want to try to make design decisions at the *most responsible moment*, the time at which a design decision has the greatest positive impact on the software system.³

3. <http://wirfs-brock.com/blog/2011/01/18/agile-architecture-myths-2-architecture-decisions-should-be-made-at-the-last-responsible-moment/>

Ideally, the last responsible moment is also the most responsible moment to decide. In practice, the most responsible moment to make a decision leaves extra time for external dependencies beyond our control, consensus building, education, and design validation. The most responsible moment is often earlier than we think it is.

Here are some questions I use to help me decide whether now is the right time to make a design decision:

- Does a lack of a decision prevent forward progress?
- Does the decision resolve a problem that cannot wait?
- Does the decision create more options or new opportunities?
- Does delaying the decision introduce significantly more risk?
- Do I understand and accept the implications of the decision?
- Do I have a clear rationale for why I am making this decision now?
- Do I have the time to undo this decision if it is wrong? Can I afford to make a mistake?

Even if we can identify the most responsible moment to decide, that doesn't mean we'll always have enough information to make a good decision. Luckily we have a cheat to help us avoid catastrophes when this situation arises. We can move things likely to change out of the architecture.

Move Design Decisions out of the Architecture

If a design decision is easy to change later, then it is no longer an architectural concern. When possible, design the architecture so that decisions likely to change are left open for downstream designers to decide.

Many of the design principles we know from programming are just plain good design principles. For example, applying SOLID principles to architecture yields many of the same benefits to architecture as they do to object-oriented design. SOLID is a mnemonic to help remember the single responsibility, open/closed, Liskov substitution, interface segregation, and dependency inversion design principles. When elements in the architecture have a single responsibility it's easier to isolate changes. Depending on abstractions and creating elements with clean interfaces creates flexibility.

There are many ways to move design decisions out of the architecture, including pluggable architectures, external configuration, self-describing data, and dynamic discovery. In each of these examples, we chose to alter the system's behavior at design time or runtime without modifying the architecture and, ideally, without adversely affecting essential quality attributes.